

SV-16 microcontroller — build plan for a 4-person team

What this is: one path from where the project is today (RTL done, simulated, bitstream builds, no hardware) to a working hand-held microcontroller board — split into roles, work packages and gates so four people can work in parallel without blocking each other.

If you have one month, not three: §13 is the dated four-week plan for a board that actually runs (chip bought week 1, PCB ordered day 10, bring-up in week 3), and §12 is the safety-net cut if any of it slips. Sections 1–11 are the full plan both are derived from.

What it is not: a schedule promise. The durations assume 4 people at roughly **8–10 hours a week each** (evenings + one weekend half-day) and no prior PCB experience. Compress or stretch §9 to your reality; the order and the gates matter more than the dates.

The deadline constraint that decides everything (read this first): the LFE5U-12F-6TG144C has a **40-week standard lead time**, but the distributor was showing **372 pieces of live stock** when this was checked (27 Sep 2026). Single pieces therefore ship in days; a back-order would take most of a year. So a one-month plan **can** put real silicon on your desk — provided you buy the chip on day 1 and buy two, and provided the two things that usually slip (fab time, 0.5 mm QFP assembly) have a fallback. §12 is the safety-net plan (software + design only); **§13 is the dated four-week plan for real hardware**, which is what you need if the review requires the board running.

Current position (measured, not estimated):

SoC / CPU / peripherals	26 RTL files, 18 test suites, 473 checks, 0 failures (make test)
FPGA fit	9,191 LUT4 = 75.7 % of the LFE5U-12F, 18/32 block RAM, 45.45 MHz against a 37.5 MHz target
Firmware	ROM monitor in flash-resident boot ROM, A/B image slots with rollback, UART update at ~5 KB/s
Hardware	none. The board is specified in BOARD.md (34 pages, India-sourced BOM) — nothing ordered or built
Biggest unknowns	first-board bring-up, the 0.5 mm TQFP-144 assembly, and FPGA availability in India

0. Finish line — pick one before you start

Level	Definition	Reached at
L1 Demo	the real monitor runs in a terminal against the whole SoC, you can upload firmware and boot it — no hardware at all	end of Phase 1 (week 3)
L2 Prototype	a physical board: powered, configured, talks on USB, accepts an A/B firmware update over the wire, blinks the LEDs, drives a motor, survives a 24 h soak	end of Phase 4 (week 12)
L3 Product	the gap list in docs/MCU_READINESS.md closed: signed updates, fault record, debug probe, measured power, EMC, enclosure	Phase 5 (open-ended)

Tell your team which level you are aiming at. Everything below targets **L2**, with L3 items parked in Phase 5 so nobody starts them early.

1. The four roles

One person owns each area — a single writer per directory. Reviewing each other's work is mandatory; editing each other's files is not.

Role	Owens (paths)	Must be good at / learn	Shows up in
P1 — Systems / RTL lead ("the chip")	rtl/, constraints/, Makefile, the bitstream flow, integration	SystemVerilog, Yosys/nextpnr, ECP5 primitives + PLL, timing, the memory map	every phase; integrator for merges
P2 — Firmware / toolchain ("the code")	firmware/, scripts/ (assembler, monitor host tool, packer), the C toolchain	Python, SV-16 ISA, UART protocols, later a C runtime; comfortable reading RTL	every phase
P3 — Hardware / PCB ("the board")	BOARD.md, board/, a new hardware/ directory with the KiCad project, the BOM, bench work	Schematic capture + layout (KiCad), ECP5 configuration rules, hand soldering/rework, DMM/scope	weeks 1-10, heaviest in 1-4
P4 — Verification / release ("the referee")	simulation/, docs/, the acceptance matrix, tags/releases, CI-ish scripts	Verilator testbenches, writing acceptance criteria that can fail, documentation discipline, scepticism	every phase; gatekeeper for "done"

Why this split: it matches the four real bottlenecks (silicon, software, board, evidence) and each role has an independent deliverable that can be shown on a Friday. Two people never wait on one file.

If someone is missing for a week: P1 and P2 can cover each other; P4 is the one role that must not be skipped, because "the design is done" is only true if someone independent reproduced it (§6).

2. The path in five phases

Phase	Weeks	Goal	Gate (must pass to leave the phase)
0 — Ground truth	0 (2-3 days)	everyone can build, test and read the project	4 laptops each run <code>make test</code> → 18 suites, 0 failures; roles agreed; parts ordered
1 — Software traction + board definition	1-3	the machine becomes touchable; the schematic exists	G1: <code>make vboard demo</code> — type ?, upload a firmware image, verify, boot it, all in a terminal. Schematic ERC-clean and reviewed
2 — Compiler + layout	3-6	you can write C for it; the PCB exists in KiCad	G2: gerbers released after review; C conformance suite green; V1 pin decision merged
3 — Fab + debug	6-9	boards on the bench; debugging works	G3: power-up checks 1-5 of <code>BOARD.md</code> §10 pass on the first assembled board
4 — Bring-up	9-12	the whole acceptance matrix passes	G4: <code>BOARD.md</code> §10 steps 6-14 signed off, v1.0 tagged, lessons written
5 — Product hardening	after	close the readiness gaps	each L3 item gets its own spec + test, like everything else here

Phase 0 — Ground truth (days, not weeks)

Who	Task	Done when
all	clone, source scripts/ sv16_venv.sh, make test, make lint, make bitstream	each person has seen the numbers in REPORT.md come out of their own machine
all	read, in this order: README.md, docs/MCU_READINESS.md, BOARD.md §1-§4, REPORT.md §8	a 30-minute meeting where each person explains one section to the others
P3	order: 2 × LFE5U-12F-6TG144C (or speed 7), an FT232H/FT2232H JTAG adapter, a ₹100 QFP-144 practice breakout, solder paste + flux + wick	tracked order numbers in the repo (hardware/ORDER.md)
P4	set up the shared evidence folder: every Friday's log, screenshot or waveform goes in one place	a link the team can open

The FPGA order is the **longest-lead item on the whole project** (40+ weeks when the distributors are empty). Place it in week 1 and treat it as the project's pacemaker.

Phase 1 (weeks 1-3) — software traction, board definition

Person	Work packages	Hand-over at the end of the phase
P1	WP2 virtual board (S0), WP8 V1 pin decision	make vboard runs the real monitor in a terminal; board/TQFP144_PINOUT.md regenerated for the final pin map
P2	WP3 upload throughput (S1), WP4 monitor protocol freeze	≥20 KB/s upload path + sv16_mon.py --fast, with fallback to today's c command
P3	WP9 schematic capture, WP10 BOM quotes + FPGA order	hardware/sv16_board/ *.kicad_sch, ERC clean, reviewed by P1
P4	WP5 host-tool unit tests, WP14 acceptance matrix draft	tests/ with golden assembler corpus + packer round-trip; docs/ACCEPTANCE.md first cut

Gate G1 demo script (run it in front of each other, not "it works on my machine"):

```
make firmware && make vboard           # terminal: monitor banner, '?' lists commands
# in the vboard terminal: upload the motor app, verify, boot, watch it run
make test                               # 18 suites, 0 failures
```

Phase 2 (weeks 3-6) — compiler and layout

Person	Work packages	Output
P1	WP8b V1 remap implementation + rebuilt bitstream, WP6 UART GDB stub (S3a)	new LPF, regenerated pin docs, gdb can attach and set a breakpoint
P2	WP5b C toolchain S2a-S2d (scripts/sv16_cc.py, crt0, printf, make cc)	make cc C=firmware/apps/ motor.c produces an image + map
P3	WP11 layout, WP12 DFM review, order boards	4-layer, DRC clean, review checklist signed by P1 and P4
P4	WP5c C conformance suite (c_tb), WP14b soak scripts	every operator/control-flow/pointer case executed in simulation with expected values

Layout cannot start before the pin map is frozen (§4) and the schematic is reviewed — that is the one hard serialisation in this project.

Phase 3 (weeks 6-9) — fab and debug

Person	Work packages	Output
P3	assembly of 2 of the 5 boards (P1 helps), bring-up steps 1-5	one board configured over JTAG, console printing
P1	WP7 JTAG debug bridge (S3b, JTAGG ER1), WP15 capacity work (S5)	OpenOCD can halt/peek/poke the CPU
P2	WP5d runtime polish, docs for the C flow	<i>firmware</i> /README section a beginner can follow
P4	WP16 docs + release process, acceptance matrix ready to execute	a checklist that a non-author can run on the board

Phase 4 (weeks 9-12) — bring-up

Everyone on one board at a time; P3 drives the bench, P1 owns the bitstream, P2 owns the firmware, P4 owns the evidence. Work the `BOARD.md` §10 list in order: rails → JTAG detect → configure → console → reset cause → LEDs → application flash → A/B update → rollback → motor → current → 24 h soak.

Expect this phase to find 3-6 real defects; that is the phase working correctly, not failing.

3. Work packages (the assignment table)

Effort is in **person-days**; "blocked by" is what must exist first.

ID	Work package	Owner	Effort	Blocked by	Definition of done
WP1	Toolchain on 4 machines, baseline green	all	0.5	—	everyone runs <code>make test</code> (18/473/0) and <code>make bitstream</code>
WP2	Virtual board (S0) : Verilator SoC + flash model + UART bridge to a terminal, <code>make vboard, sv16_mon.py --transport sim</code>	P1, P2	5	WP1	<code>make vboard</code> demo of §2 G1, plus one scripted session in <i>simulation/regression/</i>
WP3	Upload throughput (S1) : binary <code>u</code> command, block acks, optional 460800 baud, CRC per block	P2	4	WP1	≥20 KB/s measured and printed by the host tool; old <code>c</code> path still works; numbers into <i>docs/VERIFICATION.md</i>
WP4	Monitor protocol frozen: one page in <i>docs/BOOT_AND_PROGRAMMING.md</i> with every command, field and error	P2	1	WP3	P4 can implement a test client from the page alone
WP5	Host-tool unit tests : assembler golden corpus, packer round-trip, monitor framing, hex parser limits	P4	3	WP1	<code>python3 -m unittest</code> in <i>CI-ish</i> script, run by <code>make test</code>

ID	Work package	Owner	Effort	Blocked by	Definition of done
WP6	C toolchain (S2): ABI ADR, scripts/sv16_cc.py, crt0, printf, memcpy, make cc	P2	12	WP4	a real C program (motor + PWM + timer + IRQ) compiles, uploads and runs
WP7	C conformance suite: every operator, control-flow shape, pointer case, call depth	P4	5	WP6	c_tb in make test, expected values checked in simulation
WP8	Pin map freeze + V1 remap: move gpio_a[1]/[2]/[4]/[6] off the Master-SPI pads, update LPF + docs + bitstream	P1	2	—	scripts/ sv16_board_pins.py --check passes; new bitstream with ≥44 MHz Fmax; BOARD.md §6.4 rewritten
WP9	Schematic capture from BOARD.md §4–§7	P3	6	WP8	ERC clean, netlist exported, P1 + P4 review checklist signed
WP10	BOM, quotes, orders (FPGA, adapter, practice kit, tools)	P3	2	—	order numbers in hardware/ORDER.md, total ≤ budget (§8)
WP11	PCB layout (4-layer, power + FPGA + USB)	P3	8	WP9	DRC/DFM clean, gerbers + BOM + pick-and-place exported; thermal + decoupling rules of BOARD.md §11 applied
WP12	Fab + assembly plan: 5 boards, stencil, paste, hot-air, inspection	P3, P1	3	WP11	2 boards fully assembled and inspected, 3 bare as spares
WP13	Bring-up kit: bench checklist, current limits, test points, order of fitting parts	P3, P4	2	WP9	BOARD.md §10 steps 1–5 executable by someone who did not write it
WP14	Debug: S3a UART GDB stub, then S3b JTAGG bridge for OpenOCD	P1, P2	8	WP2	halt/step/peek/poke over UART; JTAG bridge demonstrated on hardware
WP15	Capacity (S5): RAM remap, more SRAM, ROM vector table	P1	5	WP2	memory map updated, loader still boots both slots, tests green

ID	Work package	Owner	Effort	Blocked by	Definition of done
WP16	Docs + release: ACCEPTANCE.md, beginner programming guide, v1.0 tag, lessons	P4	4	Phase 4	a stranger can rebuild the bitstream, flash the board and update firmware from the docs alone

Sum: ≈ 71 person-days ≈ 18 person-weeks — about 4.5 weeks of full-time work for four people, or ~11 calendar weeks at 8–10 h/week each, which is why \$2 lands on week 12. The hardware chain (WP9→WP11→WP12→bring-up) is ~25 of those days and runs mostly on P3.

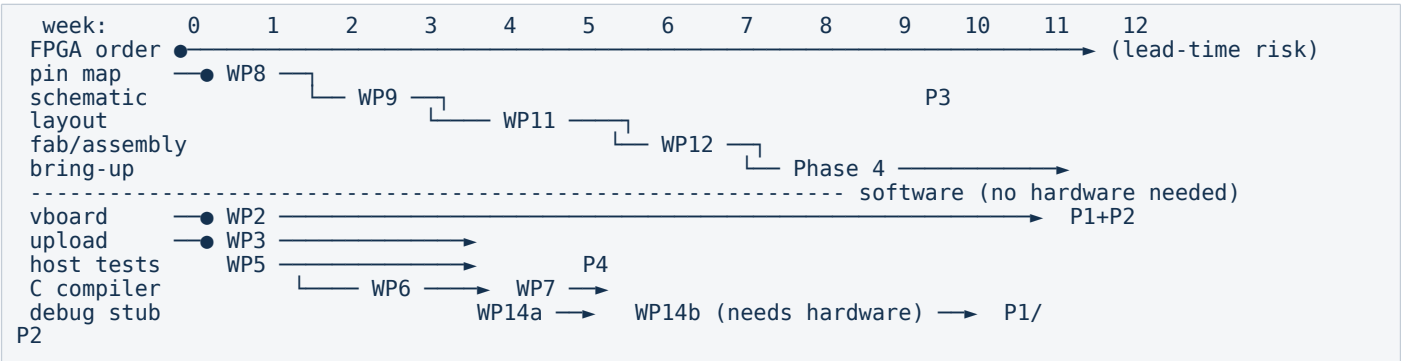
4. Interfaces that must freeze (the reason 4 people can work in parallel)

Write these down once and treat a change as a project event, not a commit:

Contract	File of record	Owner	Freeze by	If it changes
Pin map (signal → TQFP pin)	constraints/ ecp5_144tqfp.lpf + generated board/TQFP144_PINOUT.md	P1	end of week 2	P3 stops layout; regenerate docs; re-run timing
Memory map / MMIO	docs/MEMORY_MAP.md	P1	already frozen	needs an ADR + firmware change
Image + slot format	docs/ BOOT_AND_PROGRAMMING.md	P2	end of week 2	host tool, bootloader and images must move together
Monitor protocol	docs/ BOOT_AND_PROGRAMMING.md	P2	with WP4	version the change; keep the old command working
Connectors & pinouts	BOARD.md §7.5, §8	P3	before layout	reprint the schematic page + silkscreen
Power rails & sequencing	BOARD.md §4	P3	before layout	re-verify bring-up step 3
Acceptance criteria	docs/ACCEPTANCE.md	P4	before boards arrive	re-run the matrix

Rule of thumb: **anything a second person depends on gets a file of record and an owner**. If it is not written down, it is not frozen, and the other three are guessing.

5. Critical path and what can run in parallel



----- acceptance WP13 -----> WP14 matrix -----> evidence run in Phase 4 -----> P4

Serial (do not parallelise): pin map → schematic → layout → fab → bring-up. **Fully parallel:** vboard, upload throughput, host tests, C compiler, debug stub, docs. **Never** start layout before the pin map is frozen, and never order boards before the review checklist is signed — a respin costs a fortnight and ₹4,000.

6. Working agreements (this is what keeps 4 people unblocked)

1. **One writer per directory.** Reviewers comment in the PR; they do not push into someone else's WP.
2. **make test is the merge gate.** 18 suites, 0 failures, plus make lint (26 RTL files + pin-map drift check). A red build is fixed before anything new starts.
3. **No unverified numbers.** Every figure in a doc comes from a log, a datasheet or a bench measurement — this repo's existing culture (and it has already caught three invented ones).
4. **Short branches off main:** wp08-pin-remap, wp11-layout... merged by PR with one reviewer from another workstream. Delete the branch after merge.
5. **Friday demo, 20 minutes.** Each person shows one artifact: a terminal session, a schematic page, a test log, a scope screenshot. "I worked on it" is not a demo.
6. **Monday 30-minute replan.** Re-estimate the three WPs in flight; kill anything not on the critical path; move one WP if someone is blocked for more than a day.
7. **WIP limit of one.** Nobody carries two work packages at once. The board design is one WP, not five.
8. **Blame the process, then the part.** When bring-up fails, first check the schematic and the checklist, then the component — the odds are with the process.

7. What to buy in week 1 (in ₹, estimates)

Item	Why	₹
2 × FPGA (LFE5U-12F-6TG144C, or speed 7 if cheaper/in stock)	one to build, one spare — a dead FPGA otherwise costs another 40-week lead time	3,300
FT232H / FT2232H JTAG adapter	configure + debug over JTAG; also your first real "does the toolchain see the chip" test	700-1,500
Practice kit: QFP-144 breakout + spare 0.5 mm parts + flux + wick	learn drag-soldering before the ₹1,650 chip is involved	400
Solder paste + hot-air station (if not owned)	the only sane way to solder a TQFP-144 at home	2,500
Multimeter (if not owned) + USB logic analyser (₹700)	rails and UART are the two things you will debug most	1,500
Bench supply with current limit (if not owned)	bring-up without a current limit is how boards die	2,000
5 PCBs (4-layer, ~100 × 100 mm) + stencil, from JLCPCB	2 to assemble, 3 spares	3,500
Components for 5 boards (India-sourced, BOARD.md §3)	see the module-first option in BOARD.md §3.5	5,000
0603 passives assortment + wire/flux/headers/consumables	you will lose, short and re-order parts	1,200

≈ ₹21,000 with tools from scratch, ≈ ₹12,000 if your lab already has a meter, iron and air.

Everything except the FPGA can be ordered from Robu/Zbotic/Quartz; the FPGA is digikey.in / mouser.in /

8. Risks and who watches them

#	Risk	Own	Trigger to act	Mitigation
R1	FPGA not in stock (40+ week lead)	P3	no stock at digikey.in this week	accept speed grade 7 or I grade (verified: builds and closes at 49.53 MHz), or buy a Colorlight 5A-75B (₹1,500-3,000) to keep the RTL work on real silicon while waiting
R2	QFP-144 assembly fails	P3	first practice breakout has bridges you cannot clear	practice first, stencil + paste + hot air, inspect at 10×, assemble only 2 boards; fall back to BOARD.md §3.5 modules
R3	First power-up kills the FPGA	P3	1.1 V rail outside 1.045-1.155 V	bring-up step 2: fit rails, measure the 1.1 V divider, then fit U1; always use a current-limited supply
R4	Power-up order wrong on the bench	P3/P1	scope shows 1V1 before 3V3	use the PROGRAMN-hold fallback (BOARD.md §4.3); a host PWM/DTR pulse is enough
R5	Layout respin	P3/P4	review checklist incomplete	5 boards ordered, review by two people, DRC + DFM, module-first fallback for the power section
R6	Team drifts / roles blur	all	two Fridays with no demo	WIP limit, Monday replan, 20-minute demos, one writer per directory
R7	Toolchain differences between laptops	P1	"works on mine"	scripts/sv16_venv.sh pins Verilator/Yosys/nextpnr/ecppack; everyone runs make test before a PR
R8	C compiler bug ships as a "hardware" bug	P4	board misbehaves with C firmware only	WP7 conformance suite is the gate; reproduce in simulation before touching the bench

9. Compressing or stretching the plan

Situation	What to do
4 people, ~4 h/week each	keep Phases 0-2 as-is (software + schematic), then do one board per 6 weeks. Gate G1 and G2 are the value; L2 slips to ~6 months

Situation	What to do
Only 2 people	P1 takes P4's verification for RTL, P2 takes it for firmware; drop WP15 (capacity) and WP14b (JTAG bridge), and start hardware after G1
6 people	split P3's role into schematic (one person) and layout (another — layout is 8 days on its own), and give the C toolchain a second person for the runtime/printf half
No PCB ambitions	stop after G1: you still have a fully programmable MCU you can type at and flash, running on a Colorlight board or a simulator — make vboard is the deliverable
Deadline is a college review/demo	demo G1 (make vboard + upload + boot) and show BOARD.pdf + the schematic — that is a complete story without waiting for silicon

10. What to read first (per role)

Role	Read	Then
P1	docs/MEMORY_MAP.md, docs/BUS_ARCHITECTURE.md, docs/RESET_AND_CLOCK.md, ADR-021/024 (PLL)	REPORT.md §8.2 (S0, S3, S5), simulation/ to see how a testbench is written here
P2	docs/ISA.md, docs/BOOT_AND_PROGRAMMING.md, firmware/monitor/monitor.s	scripts/sv16_mon.py, then the S1/S2 items in REPORT.md §8.2
P3	BOARD.md (all of it), board/TQFP144_PINOUT.md, docs/SYNTHESIS_AND_DEPLOYMENT.md (pin map)	ECP5 sysCONFIG (TN1260) and the KiCad ECP5 design rules; BOARD.md §11 before drawing anything
P4	docs/VERIFICATION.md, every testbench in simulation/, docs/MCU_READINESS.md	write the acceptance matrix from BOARD.md §10 before the boards arrive

11. Tell me these five things and I will re-cut the plan

- Hours per week per person, and the calendar deadline (if any).
- Who has what: soldering iron/hot air, DMM, scope, bench supply, FPGA/Arduino experience, KiCad or any CAD experience, C/Python level.
- Budget ceiling for parts + tools (§7 is ₹12–21k with/without tools).
- Do you have a college lab or a makerspace with a reflow oven, microscope or assembly service?
- Do you want the first board to be the **module-first** version (BOARD.md §3.5, safer, bigger) or the all-soldered version (smaller, riskier)?

With those answers this document becomes a dated sprint plan with names next to each WP — and I can cut it down to a one-page chart for your group.

12. One month, four people — the realistic cut

§12 assumes the chip could not be obtained in time. It can (§13): the distributor had 372 pieces of live stock on 27 Sep 2026. Treat §12 as the fallback, and read §13 if you are buying the chip — then come back here for the capacity arithmetic and the cut list.

You said the deadline is one month. Here is the arithmetic first, then the plan.

Capacity. Four people × four weeks:

Effort per person per week	Total person-hours	= person-days (8 h)
12 h (a normal college week)	192	24
20 h (a real crunch)	320	40
25 h (exam-week style, unsustainable)	400	50

The full plan in §1–§11 is **≈71 person-days**. A month gives you 24–50 of them. So the first decision is not how to do everything, it is **what to leave out**, and the answer is decided by one fact:

If you cannot lay hands on the FPGA itself, four weeks gets you a working software system and a finished, orderable board design — but not a board that runs. (The reorder lead time is 40 weeks; the plan for actually getting the chip is §13.)

Everything below works with that fact instead of pretending otherwise.

12.1 What "done in one month" means — freeze this sentence today

On demo day the real SV-16 SoC runs in a terminal, I can upload a new firmware image over the same UART protocol a real board would use, verify it, boot it, roll it back — and next to that I show the finished PCB design (schematic, layout, gerbers, 3D render, India-sourced BOM) that is one order away from being manufactured.

That is a complete story: the software is real (it is the same RTL, same monitor, same protocol, same memory map as the silicon would run), and the hardware is designed (it is the actual deliverable of P3's four weeks). What it is not is a photo of a soldered board with a blinking LED — say that out loud in the first 30 seconds of your demo and you control the conversation, instead of being asked why there is no board on the table at minute 5.

Status change (27 Sep 2026): this was written when the chip looked unobtainable in a month. It is obtainable (§13). §12.1 is therefore now the **fallback** the project falls back to if fab or assembly slips — not the target. Keep it in your pocket; aim at §13.

Two levels, pick one now:

	Level M1 — recommended	Level M2 — only if you can borrow silicon
Demo	§12.1 as written (software live + board design)	M1 plus the SoC running on a real ECP5 board that already exists (§12.3)
Cost	₹3,000–6,000 (PCB + parts ordered, tools shared)	+ ₹1,500–3,000 for a ready-made ECP5 board
Risk	low: no dependency on any chip arriving	medium: needs 3–4 days of P1+P2 time for a new pin constraint file
I'd choose it	yes, for a review or evaluation	if the reviewer explicitly demands "real hardware running"

12.2 The four weeks

Week	P1 — systems/RTL	P2 — firmware/tooling	P3 — hardware	P4 — verification	Gate at the end
1	virtual board WP2: Verilator SoC + flash model + UART bridge, make vboard	upload path WP3: binary u command + sv16_mon.py --fast; monitor protocol WP4	schematic WP9 from BOARD.md §4–§7; place the parts + PCB order (§12.5)	acceptance matrix WP14 + host-tool tests WP5	G1: make vboard shows the monitor, an image uploads, verifies and boots — on all four laptops. Schematic ERC-clean

Week	P1 — systems/RTL	P2 — firmware/tooling	P3 — hardware	P4 — verification	Gate at the end
2	pin-map freeze WP8 (V1 remap) + bitstream; start the ready-made-board LPF if going M2	firmware app for the demo; host tool docs; sim transport used by P4's tests	layout WP11, DRC, order the PCB by day 10 (it needs ~7 days to arrive)	tests green in <code>make test</code> ; evidence folder started; <code>ACCEPTANCE.md</code> written	G2: gerbers ordered; upload ≥20 KB/s; two images (A/B) demonstrated with rollback
3	regression + timing; M2 only: port the LPF to the ready-made board and get the SoC running on real silicon	demo firmware polished (console + PWM + motor sequence in the simulator)	practice-solder the ₹100 QFP breakout; when the PCB lands: assemble 1-2 boards; verify rails, oscillator, USB enumerate (bring-up steps 1-5 without fitting the FPGA)	run the acceptance matrix, collect logs + waveforms for the report	G3: board assembled and its rails measured; demo firmware runs end-to-end
4	buffer: fix whatever week 3 broke; freeze the bitstream	freeze the toolchain; write the programming guide	3D render + gerber screenshots + BOM cost table for the review	rehearse the demo twice; tag the release; write the one-page summary	G4: demo rehearsed twice, video recorded, release tagged, docs updated

Week 4 is deliberately half-empty. A four-week plan with no buffer is a three-week plan with a public failure at the end.

12.3 If the reviewer wants real silicon (level M2)

You cannot wait for a LFE5U-12F-6TG144C — but you can hold an ECP5 in a week:

Option	Price (₹)	What you get	Work required
Colorlight 5A-75B (LED-panel controller, has an LFE5U-25F-6BG381C)	1,500–3,000 (Amazon.in / eBay.in)	a real ECP5, JTAG already reverse-engineered (q3k/chubby75), 2 × 8 MB flash, LEDs, Ethernet	3–4 person-days by P1+P2: new <code>.lpf</code> for BG381, blink test, UART on its pins, then the full monitor
Any ECP5 dev board your college lab owns (ECP5-EVN, ULX3S, a custom board)	0	the best case — ask your HOD this week, not week 3	1–2 person-days: new <code>.lpf</code> + pin docs, same as above
A second-hand ECP5 board	2,000–6,000	varies	as above

Rules for M2: the design itself is **unchanged** (same RTL, same bitstream flow, `make bitstream` with a different `--lpf`), and `BOARD.md` §6's TQFP-144 pin map stays the reference for your board. Do not let M2 leak into the custom PCB — the two pin maps are different by definition, and mixing them is how projects lose a week.

If M2 is not possible: **do not apologise for it**. Show a timing report of the design placed and routed for the real chip, and the board design that would carry it. That is more evidence than a blinking LED.

12.4 What to cut, in order

Cut from the top until the remaining list fits your real hours.

Priority	Item	Why
Must not cut	WP2 virtual board, WP3 upload, WP4 protocol, WP9 schematic, WP11 layout, WP14 acceptance	these are the demo and the deliverable
Cut first	WP15 capacity/RAM work, S3b JTAG bridge, S4 keyed MAC	interesting, invisible on demo day

Priority	Item	Why
Cut second	WP6/WP7 the C compiler + its conformance suite	17 person-days — a month of one person, for something an evaluator will not ask about if you never mention it. Make it your "future work" slide
Cut third	WP8 V1 pin remap	the current pin map works; remapping only makes the next board nicer
Keep if you can	WP3 at 460800 baud, second firmware slot demo, <code>BOARD.md</code> cost table	cheap, and they read as "engineering" to a reviewer

If you keep the C compiler, you lose either the PCB layout or the demo polish. Pick one, write it down, and do not revisit it in week 3.

12.5 Week 1, day by day (the week that decides the month)

Day	Everyone	Notes
1	make test on all four machines (18 suites, 0 failures); agree §12.1 in writing; assign P1–P4	nobody writes code tomorrow who cannot build today
2	P1: vboard skeleton. P2: U-command design + framing. P3: schematic — power tree first (<code>BOARD.md</code> §4.3), then FPGA core. P4: acceptance matrix from <code>BOARD.md</code> §10	P3 places the PCB order on day 10 at the latest — check fab lead time on day 2, not day 20
3	P1: UART bridge + flash model. P2: host-side block protocol. P3: connectors + USB + flash. P4: host-tool unit tests	mid-week check: can P4 talk to the vboard UART yet?
4	P1+P2 integrate: real monitor binary in the vboard, terminal attached. P3: ERC pass. P4: first acceptance run	this is the day the project becomes demoable
5	G1 demo, 20 minutes, all four laptops. P3: BOM + orders (tools, breakout, modules). P4: evidence folder	if G1 slips to day 7, cut WP8 immediately
6–7	buffer + ordering. Anything not ordered by day 7 does not exist in this project	weekend is when Indian vendors actually get ordered from

12.6 Demo-day script (5 minutes, rehearse it twice)

- 30 s — what it is:** "an ECP5 microcontroller: RISC CPU, 16 KB RAM, flash boot, A/B firmware slots, UART monitor, PWM/GPIO/timer/WDT — 18 test suites, 473 checks, 0 failures."
- 2 min — live:** terminal → monitor banner → ? → X (erase) → upload an image → V (verify) → B (boot) → the application prints over the same console. Then break it on purpose: upload a deliberately corrupt image and let the **rollback** save the board. This is the moment that makes it look like a product.
- 1 min — the hardware:** schematic page, layout, 3D render, gerbers, the India-sourced BOM with prices, and `BOARD.pdf` on the table.
- 1 min — the honest part:** the FPGA could not be secured in time (40-week reorder lead time), so the board is designed and ordered, rails verified, bring-up checklist ready; here is the Colorlight plan (or the lab board) for real silicon.
- 30 s — next month:** the C compiler, JTAG debug, RAM expansion — from `REPORT.md` §8.

Record it on video on day 27. Live demos fail; a video in your pocket does not.

12.7 One-month risk list

Risk	Trigger	What you do
PCB late or held at customs	not shipped by day 14	it does not matter: the software demo carries the review; show the gerbers and the render
A teammate's week collapses (exams, illness)	anyone misses two days	P1+P2 own the demo path; P3+P4 own the evidence. Nobody else's work is on the critical path
vboard harder than expected	not running by day 4	fall back: run the monitor in the existing Verilator SoC testbench with a UART bridge to a terminal — less pretty, same evidence. <code>soc_boot_tb</code> already proves the flow in simulation
Upload speed work drags on	no <code>u</code> command by day 4	demo the existing <code>c</code> command; say "20 KB/s is the next revision"
FPGA/parts money runs out	—	the PCB can be ordered bare (₹1,500) and populated later; the software deliverable costs ₹0
Scope creeps in week 3 ("let's add...")	anyone says "while we're here"	§12.4 is frozen; new ideas go on the "next month" slide
Review demands hardware running	—	M2 (§12.3): buy or borrow an ECP5 board in week 1, port the LPF, and show the same demo on real silicon

12.8 The one-page version

Week 1: make the machine touchable (make `vboard` + upload + boot) and finish the schematic — order the PCB and parts. **Week 2:** layout, DRC, order (day 10); pin map frozen; A/B rollback demoed. **Week 3:** assemble what arrives, verify rails; polish the demo firmware; run the acceptance matrix. **Week 4:** buffer, video, release tag, one-page summary. **Not in this month:** the C compiler, the extra RAM work, the JTAG bridge, and the FPGA chip itself — unless you secure distributor stock on day 1 (§13), in which case the board runs and this section is only the fallback.

13. Four weeks with real hardware — the dated plan

You said three things: **the board must run**, you can work **~20 h per person per week** (≈ 40 person-days total), and you can buy the chip and the components. Those three facts make the aggressive plan viable — with two dates in it that cannot move.

Dates below assume D1 = Monday 28 Sep 2026 and a review on D28 = Sunday 25 Oct 2026. Slide the whole table if your start or review date differs; keep the gaps (D10 and D17) exactly as they are, because they are derived from fab lead time and assembly risk, not from optimism.

13.1 The critical path with hard dates

	Date	What must happen	If it slips
D1	Mon 28 Sep	Order everything, today: 2 × LFE5U-12F-6TG144C (digkey.in — 2 because a dead FPGA otherwise ends the project), FT232H/FT232H JTAG adapter, ₹100 QFP-144 practice breakout, and the whole <code>BOARD.md</code> §3 BOM from Robu/Hubtronics/Sharvie/iFutureTech. Create <code>hardware/ORDER.md</code> with order numbers	every day of delay here is a day off the end

	Date	What must happen	If it slips
D2-D5	Tue 30 Sep – Fri 02 Oct	P3: schematic (power tree → FPGA → USB → flash → connectors, using <code>BOARD.md</code> §6's net table and <code>constraints/ecp5_144tqfp.lpf</code> as the pin truth). P1: make <code>vboard</code> . P2: upload path. P4: acceptance matrix. Chip usually lands D4-D5 — verify it is genuine (Lattice marking, digikey packing) before touching it	schematic slipping past D5 costs layout days
D5	Fri 02 Oct	Gate S: schematic ERC-clean and reviewed by P1+P4. If the chip did not ship: order a Colorlight 5A-75B today as the demo hedge (\$12.3)	—
D6-D10	Sat 04 – Wed 07 Oct	P3: layout — 4-layer, follow <code>BOARD.md</code> §11 (decoupling, plane splits, via-per-pin). P1: review nets against the LPF. P4: DRC + DFM checklist. Also this week: practice-solder the ₹100 breakout	layout is 8 person-days in 5 calendar days — protect it
D10	Wed 07 Oct	HARD DATE — order the PCB (5 pcs, stencil, ~₹3,000, JLCPCB). ~7 days fab + shipping	order on D11 and you lose the demo, not just a day
D10-D17	Thu 08 – Wed 14 Oct	Software sprint while the board is in fab: make <code>vboard</code> demoable, upload ≥20 KB/s, A/B rollback in simulation, host-tool tests green. P3: assembly prep — stencil, paste, hot air, find a local rework shop as backup (₹500–1,500 to have a QFP placed professionally)	—
D17	Wed 14 Oct	Boards in hand. Inspect and assemble 2 (fit U1 last). Practice parts first	if fab or customs slips: demo path = Colorlight (S-switch), board becomes "arriving"
D19	Fri 16 Oct	Rails verified with U1 not fitted — 3V3, 1V1 at 1.10 V ±3 %, 2V5, sequencing on a scope (<code>BOARD.md</code> §4.3 timeline), oscillator running, USB enumerates (CH340C)	this is where a layout error is cheapest to find and fix
D20-D21	Sat 17 – Sun 18 Oct	Fit U1. JTAG detect (<code>openFPGALoader --detect</code>), then <code>make prog FPGA_PART=LFE5U-12F</code> . DONE LED lights, console prints the monitor banner	if JTAG cannot see the chip: reflow the QFP, check all four rails again — do not start rework roulette without a plan
D22-D24	Mon 19 – Wed 21 Oct	On hardware: ? menu, flash ID, upload a firmware image over UART, verify, boot from flash, then the deliberate corrupt-image rollback	if the console works but flash does not, demo the RAM path and say so

	Date	What must happen	If it slips
D25-D27	Thu 22 – Sat 24 Oct	Run the acceptance matrix on the real board, record the video, tag the release, update <code>REPORT.md</code> with measured results	video on D27 is not optional
D28	Sun 25 Oct	Buffer / demo day	—

Total spend for this path: ₹15,000-25,000 — chip $\times 2 \approx ₹4,000$ landed, JTAG adapter ₹700-1,500, PCB + stencil ₹3,000, components ₹5,000, Colorlight hedge ₹2,000-5,000, rework-shop contingency ₹1,000, tools if you do not own hot air/meter/current-limited supply ₹4,500+. If that is too much, cut the hedge and the second chip last — they are the two things that convert "we nearly made it" into "we made it".

13.2 Three work packages per person, in order (≈ 40 person-days)

	P1 — systems/RTL	P2 — firmware/tooling	P3 — hardware	P4 — verification
Priority 1	make vboard (5 d)	upload path ≥ 20 KB/s (4 d)	schematic (6 d)	acceptance matrix (3 d)
Priority 2	LPF $\leftrightarrow$ layout net review (2 d)	monitor protocol + host docs (2 d)	layout + DRC (8 d)	host-tool tests (3 d)
Priority 3	bring-up support: JTAG, config, timing (4 d)	demo firmware + rollback test (3 d)	assembly + rails verification (5 d)	run matrix on hardware + evidence (4 d)
Cut first	capacity/RAM work	C compiler	3D render polish	pretty docs
Total	11 d	9 d	19 d	10 d

P3 carries 19 of the 40 person-days. That is the honest shape of a hardware deadline: **the board is the project**. If someone has spare time in week 1, it goes to P3 (schematic), not into a new feature.

13.3 The three switch points

Write these into the plan now so nobody has to decide under pressure:

Switch	When	Condition	Action
S-chip	D5	chip not shipped, or stock gone	order a Colorlight 5A-75B from Amazon.in / a local LED-panel supplier (same week), port the LPF to BG381 in 3-4 d; the custom board still gets built — it just stops being the demo path
S-fab	D17	boards not in hand	demo runs on the Colorlight; show gerbers + render + the fab tracking page; assemble and bring up the custom board later
S-assembly	D21	both assembled boards fail to configure	pay a local rework shop to redo the QFP on board #3 (₹500-1,500), keep the Colorlight as the live demo, and present the failure analysis — a documented failure with a fix beats a silent one

Never let the demo depend on one board. Two assembled boards, one alternative board, one recorded video — that is the entire risk story of a one-month hardware deadline.

13.4 Why this can work (and what it would take to break it)

Working for you:

- The RTL, the ISA, the memory map, the monitor, the bootloader and the flash driver already exist and are verified — 473 checks. There is no design work left in the chip.
- The pin map **is already written** as `constraints/ecp5_144tqfp.lpf` (52 constrained pins) and printed as the net-by-net table in `BOARD.md §6/Appendix B`. Layout is a translation job, not an invention.
- The BOM is India-sourced with prices and stock already recorded (`BOARD.md §3`) — nobody spends week 1 emailing distributors.
- The board is specified down to the passives (`BOARD.md §3.3, §4–§7`): five sheets of connection diagrams, bring-up steps 1–14, and a five-condition rule for the Master-SPI pins.

Working against you (in order of how likely it is to hurt):

1. **First-time 0.5 mm QFP assembly.** Practice on the ₹100 breakout; use stencil + paste + hot air; inspect at 10×; and have the rework shop's number before you need it.
2. **Layout complexity with 8 person-days available.** Mitigate by following `BOARD.md §11` literally and by not inventing footprints — and by keeping the power section on the module-first option (`§3.5`) if the schematic review reveals trouble.
3. **Two-person dependency on the critical path** (P3 layout, P1 bring-up). Both must be contactable in week 2 and week 3 — a two-day silence in either role costs the month.
4. **The review date moving earlier.** If it moves, cut the custom board to "awaiting fab" and demo on the Colorlight without hesitation.

13.5 What to show on demo day (real-hardware version)

1. **30 s:** what it is — ECP5 microcontroller, RISC CPU, flash boot, A/B slots, UART monitor, 473 checks.
2. **2 min:** power the board on camera → DONE LED → console banner → ? → upload a new firmware image → verify → boot → application prints. Then upload a corrupt image and let the rollback recover it.
3. **1 min:** the board itself in hand: with the schematic page, the layout, and the net table that proves every pin came from the constraint file.
4. **1 min:** the honest engineering: rails measured against `BOARD.md §4.3`'s timeline, JTAG chain, flash ID, and where the design would go next (C compiler, JTAG debug, RAM expansion — `REPORT.md §8`).
5. **30 s:** the plan that survives: if this exact board had failed, here is the fallback (Colorlight) and here is the evidence of the failure — that is what "engineering" sounds like.

14. "Can't we just upload the .sv code and it runs?" — what actually has to happen

Short answer: **no**. The FPGA cannot read SystemVerilog any more than a phone can read C. The `.sv` files are the design of the hardware; they have to be compiled into a bitstream, and then two separate memories have to be programmed. Four artifacts, four steps, two different flash chips:

#	Artifact	Produced by	Written into	Command	Survives power cycle
1	Bitstream , <code>build/sv16_top.bit</code> (293 KB)	your laptop: Yosys → nextpnr → ecppack over the 26 <code>.sv</code> files (make bitstream, a few minutes)	U1's configuration SRAM, over JTAG	<code>make prog</code> <code>CABLE=ft232</code>	no — gone when power drops

#	Artifact	Produced by	Written into	Command	Survives power cycle
2	the same bitstream, permanently	—	U2 , the configuration flash, through the FPGA's MSPI port	make prog-flash CABLE=ft232	yes — this is the step that turns the board into a microcontroller at power-up
3	Firmware image (SV-16 hex + header + CRC)	your laptop: make app, make slot-image SLOT=1	U3 , the application flash, over UART through the monitor	make upload-slot SLOT=1 PORT=/dev/ ttyUSB0 → make mon-boot → make commit	yes
4	the monitor (boot ROM)	your laptop: make rom — it is compiled into artifact 1	nowhere separate — it lives inside the bitstream	rebuild and re-flash 1 / 2	yes (it is the configuration)

Three things people trip over:

1. **.sv is not a program.** Artifact 1 defines what the chip is; artifact 3 is what it runs. They are different files in different flash chips (U2 vs U3) written by different tools (JTAG vs UART).
2. **The monitor cannot be updated over the serial port.** It is hardened into the bitstream, so a monitor bug means rebuilding the bitstream and re-flashing U2 over JTAG — not an upload. (Applications, by contrast, can always be replaced over the console.)
3. **U2 and U3 are the same part with different jobs.** BOARD.md §9 has the table, and §9.1 has these four flows in full.

14.1 Day of first power, in order

```
make bitstream # 1. on the laptop
# 2. rails only, U1 not fitted, current-limited supply (BOARD.md §10 steps 1-3)
# 3. fit U1 ->
openFPGALoader --detect # should report the ECP5 IDCODE
make prog CABLE=ft232 # 4. volatile config: safe, a power cycle undoes it
make mon-term PORT=/dev/ttyUSB0 # 5. terminal at 115200 -> the monitor banner
make prog-flash CABLE=ft232 # 6. write U2, then power-cycle with no cable: banner again
make upload-slot SLOT=1 PORT=/dev/ttyUSB0 # 7. field update: upload -> mon-boot -> commit
```

Step 4 before step 6 on purpose: if the bitstream is wrong, a power cycle erases the mistake. Only once the design behaves do you write it into U2.

14.3 One command that checks all four (run it before you program the board)

make release builds the firmware and the bitstream, then verifies everything about the four artifacts that can be verified without hardware, writes a manifest with their SHA-256 hashes to build/release/, and prints the programming sequence:

```
make release # osc variant (25 MHz), the board's oscillator
make release CLKSRC=pll PLLMHZ=37.5 # if the team prefers the 37.5 MHz PLL build
scripts/sv16_release.py --twice # also rebuild twice and compare hashes
```

What it checks, and why each one has burned somebody before:

Check	Why it matters
The boot ROM re-assembles byte-for-byte from firmware/monitor/monitor.s	a stale ROM means the bitstream carries an old monitor — you would debug the wrong code on the bench
The bitstream is newer than the ROM	same reason, caught by timestamp instead of content
nextpnr's reported Fmax meets the constraint (PASS/FAIL line)	a build that failed timing still produces a bitstream, and it is the one that works most of the time
The device budget check passes against the datasheet limits	proves the design still fits the LFE5U-12F you are soldering

Check	Why it matters
Every image's header CRC and payload CRC	a truncated upload is silent otherwise; the loader would reject it at boot and you would blame the flash
The payload fits the 32 KB slot, and the slot record state	tells you whether an image can roll back (PENDING) or overwrite without protection (no record)
SHA-256 of every artifact	so "which file did we flash?" is a one-line answer, not a memory test

Result on this tree (verified today, CLKSRC=osc, 25 MHz): 10 checks passed, 0 failed — ROM 1115/2048 words with 933 free, bitstream 295,665 bytes, timing 36.29 MHz against the 25 MHz constraint, 9,407 LUT4 (77.5 %) of the device, images `motor_test_img.hex`, `app_slot1_img.hex` (slot state **pending**) and `wdt_hang_img.hex` all with valid CRCs. With `--twice`, **the bitstream reproduced byte for byte across two complete builds** (identical SHA-256) — so a bitstream hash in the manifest really does identify a design revision, which is what makes the release record worth keeping.

Note what this does not do: it cannot tell you the bitstream will configure your board, or that the soldering is good. Those are hardware facts and they are what the 1-3 day bring-up is for (§14.2).

14.2 "And it will run" — the honest caveat

The logic is verified: 18 test suites, 473 checks, 0 failures; timing closes at 45.45 MHz against a 37.5 MHz target; the design fits the device with 24 % of the LUTs spare. What has never happened is **this board existing**. Bring-up is its own phase — budget **1-3 days** — and the likely failures are physical, not logical:

Symptom	Usual cause	Where to look
<code>openFPGALoader --detect</code> finds nothing	QFP bridges, or a rail missing/off-spec	<code>BOARD.md</code> §10 steps 2-4; recheck all four rails and the JTAG header wiring
Detects, but configuration never completes (DONE stays low)	CFG strapping, or a bad bitstream/config-flash wiring	<code>BOARD.md</code> §10 steps 5-7; check <code>CFG[2:0] = 010</code> and R13-R15 pull-ups
Configures, but no banner	wrong baud, RX held low, or a UART pin swap	<code>docs/BOOT_AND_PROGRAMMING.md</code> §7 symptom table; try <code>make mon-term</code> at 115200 and check TX/RX
Banner appears, flash commands fail	U3 wiring / CS or MISO swapped	flash ID first (<code>BOARD.md</code> §10 step 9), then the upload
It ran once and now does nothing after power-up	U2 was written before the design was proven	<code>JTAG make prog</code> restores it — that is why step 4 comes first

So the accurate version of your sentence is: "once the board is built and the rails check out, we compile the design to a bitstream, flash that into U2, upload our firmware over UART — and then it runs, after a bring-up debugging pass whose size depends on how good the soldering was."

Revision 1.3 — written against commit `dff0001` (project state: RTL complete, 473 checks green, bitstream builds, no hardware). §12 is the one-month cut; **§13 is the dated four-week hardware plan** (the deadline requires the board to run, the team has ~20 h/week each, and the FPGA is purchasable — 372 pieces of live distributor stock on 27 Sep 2026; the 40-week figure is the reorder lead time); §14 answers "can't we just upload the .sv code?" (and §14.3 is the release checker) (no — bitstream, config flash, application flash, and the monitor inside the bitstream). §1-§11 remain the full plan. Effort estimates are for a first-time hardware team — the figures taken from real measurements are the 473 checks and the 49.53 MHz speed-7 timing run.